LAB 4 - Advanced API Integration: File Upload, External APIs & JWT Authentication

Course: Web Application and Database Development Practicum (Backend & Database)

1. Core Theory Summary

1.1 Same-Origin Policy (SOP) & CORS Configuration in Express

Web browsers enforce the **Same-Origin Policy (SOP)**: JavaScript running on one origin may not read responses from another origin. An **origin** is the combination **Protocol** + **Domain** + **Port**, so `http://localhost:3000` and `http://localhost:5001` are two different origins.

To let a Web UI (client) call a RESTful API hosted on a different port or domain, the Express backend must enable **CORS (Cross-Origin Resource Sharing)** by sending the right HTTP response headers. Note that CORS is enforced by the **browser** only: Postman and server-to-server calls are not affected.

HTTP Response Header	Purpose	Common Value
`Access-Control-Allow-Origin`	Which origins may access the API's responses.	`*` (any origin) or `http://localhost:3000`
`Access-Control-Allow-Methods`	Which HTTP methods the client may use.	`GET, POST, PUT, PATCH, DELETE, OPTIONS`
`Access-Control-Allow-Headers`	Which request headers the client may send.	`Content-Type, Authorization`

Example: CORS for specific origins (production style)

```
const cors = require('cors');

app.use(cors({
  origin: ['https://your-production-frontend.com', 'http://localhost:3000'],
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true // only needed when the browser must send cookies
}));
```

> **Preflight requests**
>
> For "non-simple" requests (e.g. `PUT`/`DELETE`, or a JSON body with `Content-Type: application/json`, or an `Authorization` header) the browser first sends an automatic **OPTIONS** request to ask for permission. If that preflight response does not carry the right CORS headers, the real request is never sent and the Console shows a **CORS policy** error.

1.2 Web UI ↔ Backend Communication via Fetch API & Axios

Client-side JavaScript talks to a RESTful API asynchronously using the browser's built-in **Fetch API** or the **axios** library. Data is usually sent as **application/json** (a JSON string) or as **multipart/form-data** (when a file is sent together with other fields).

Always use **async/await** inside a **try…catch** block. Remember that `fetch()` only rejects on a **network** failure: a 4xx/5xx response still resolves, so check `response.ok` yourself.

Example: fetch with error handling

```
async function loadProducts() {
  try {
    const response = await fetch('http://localhost:5001/api/v1/products');
    const result = await response.json();
    if (!response.ok) throw new Error(result.message || `HTTP ${response.status}`);
    return result.data;
  } catch (error) {
    console.error('Could not load products:', error.message);
    throw error;
  }
}
```

1.3 Calling External (Third-Party) APIs from the Backend with Axios

Besides serving its own API, a backend often acts as a **client** itself, calling a third-party service such as a weather provider, a payment gateway or an SMS gateway. This call is **server-to-server**, so CORS does not apply and any secret API keys stay on the server. On the server, axios is the usual choice: it parses JSON automatically, supports interceptors, and makes it easy to set a timeout.

Example template

```
const axios = require('axios');

async function getExchangeRate(base) {
  try {
    const response = await axios.get(`https://api.example.com/rates/${base}`, {
      timeout: 5000 // abort the call if it takes longer than 5 seconds
    });
    return response.data;
  } catch (error) {
    if (error.response) {
      // The external API answered with an error status (4xx/5xx)
      console.error('External API error:', error.response.status, error.response.data);
    } else if (error.request) {
      // The request was sent but no answer came back (timeout, DNS, network...)
      console.error('No response from external API:', error.message);
    } else {
      console.error('Unexpected axios error:', error.message);
    }
    throw error;
  }
}
```

1.4 Debugging API Calls with Browser DevTools

Browser DevTools (Chrome / Edge: press **F12** or **Ctrl+Shift+I**, **Cmd+Option+I** on Mac) is the main debugging tool of every fullstack developer. When a Web UI's call to an API fails, check these places in order:

1. **Network panel:** filter by **Fetch/XHR**. For each request, read the **status code** (200, 400, 401, 403, 404, 500), the **method** and the time it took. A red row with no status usually means a CORS or network error.
2. **Headers tab** (click a request): check the request URL, the **request headers** (Content-Type, Authorization) and the **response headers** (Access-Control-Allow-Origin).
3. **Payload tab:** confirm the JSON or form-data the client actually sent.
4. **Preview / Response tab:** read what the server sent back, including its error message.
5. **Console panel:** read the red error messages (CORS policy errors, uncaught promise rejections, "Failed to fetch") and click the file link to jump to the line of code.

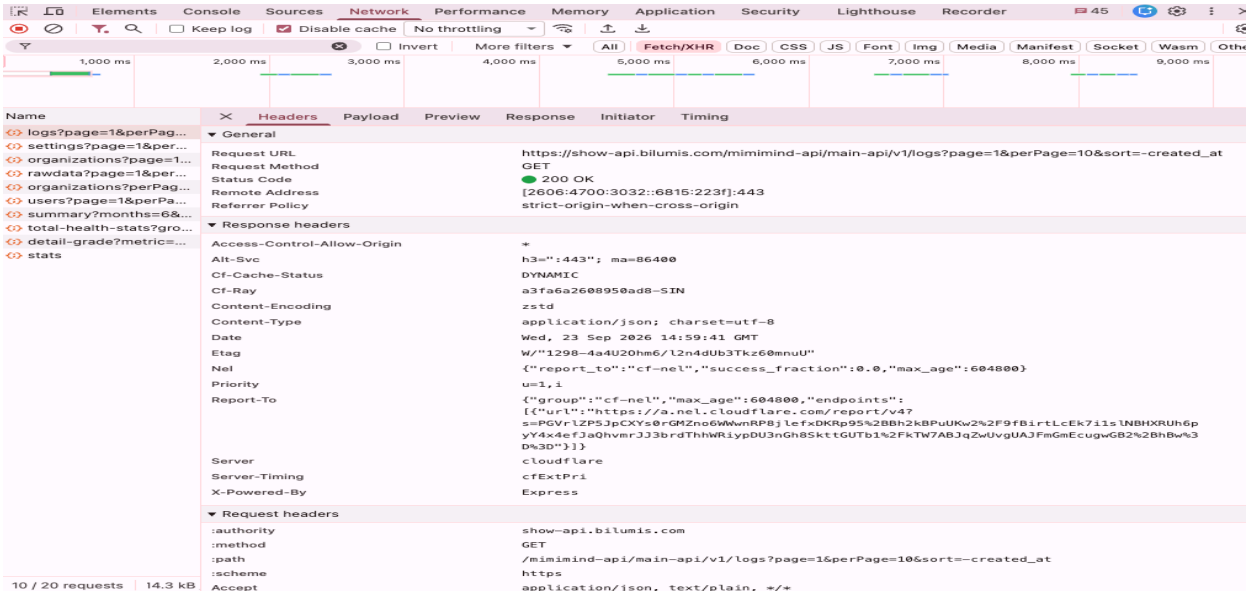


Figure 1.1: The Network panel with the Headers tab of a selected request

1.5 File Uploads & Downloads with Multer and Static File Serving

express.json() and express.urlencoded() only parse JSON and URL-encoded bodies. A file sent from a form with **enctype="multipart/form-data"** (or a FormData object in fetch) needs the **multer** package to parse the file and save it, for example to disk (**DiskStorage**).

So that a client can open an uploaded image directly by URL (e.g. http://localhost:5001/uploads/avatar-123.jpg), the backend serves the folder as static files: app.use('/uploads', express.static('uploads')).

express.static lets the browser display the file **inline** (an image preview). To force a **"Save As" download** instead, which is useful for documents, use res.download(filePath): it sets the **Content-Disposition: attachment** response header.

1.6 Session vs JWT: Concepts & Trade-offs

Both sessions and **JSON Web Tokens (JWT)** solve the same problem: remembering that a user is logged in across many requests. Their trade-offs are very different:

Criteria	Session (server-side, cookie-based)	JWT (stateless token)
Where state lives	On the server (memory, Redis, DB); the client only holds a session ID cookie	Inside the token itself, held by the client
Statefulness	Stateful: the server looks the session up on every request	Stateless: the server only verifies the signature
Scaling across servers	Needs a shared session store (e.g. Redis) behind a load balancer	Scales horizontally with no shared state
Revoking access early	Easy: delete the session on the server	Hard: a valid token works until it expires, unless you add a blocklist
Typical use case	Traditional server-rendered web apps	APIs used by SPAs, mobile apps and third-party clients

1.7 Hashing Passwords with bcryptjs

Passwords must **never** be stored as plain text. **bcryptjs** is a JavaScript implementation of the **bcrypt** hashing algorithm. Hashing is one-way: it is easy to hash a password but practically impossible to turn the hash back into the password. bcrypt adds a new random **salt** to every hash, so two identical passwords produce different hashes. The **salt rounds** value is the cost factor: each +1 doubles the time needed to compute (and to brute-force) a hash; **10** is a common choice.

Hashing and comparing with bcryptjs

```
const bcrypt = require('bcryptjs');

// Register: hash the plain-text password before saving it
const passwordHash = await bcrypt.hash(plainPassword, 10); // 10 salt rounds

// Login: compare the submitted password with the stored hash
const isMatch = await bcrypt.compare(plainPassword, user.passwordHash);
```

1.8 Issuing and Verifying a JWT in Express

A JWT has three parts separated by dots: **header.payload.signature**. The payload (e.g. the user's id and type) is only **Base64URL-encoded, not encrypted**: anyone can read it, so never put a password or other secret in it. The **signature**, made with a secret key that only the server knows, is what proves the token has not been changed.

1. **Login:** after checking the password, the server signs a token with jwt.sign() and returns it.
2. **Every later request:** the client sends it in the header Authorization: Bearer <token>.
3. **Protected route:** the server checks it with jwt.verify() before running the controller (the same middleware pattern you used in Lab 3).

Example with the jsonwebtoken package

```
const jwt = require('jsonwebtoken');

// 1. Login: issue a token that expires after one day
const token = jwt.sign(
  { id: user.id, type: user.type },
  process.env.JWT_SECRET,
  { expiresIn: '1d' }
);

// 2. Protect a route: read and verify "Authorization: Bearer <token>"
function protect(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) {
    return res.status(401).json({ success: false, message: 'Please log in first' });
  }
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET); // { id, type, iat, exp }
    next();
  } catch (error) {
    return res.status(401).json({ success: false, message: 'Invalid or expired token' });
  }
}

// 3. Check the role after the token is verified
function adminOnly(req, res, next) {
  if (req.user.type !== 'admin') {
    return res.status(403).json({ success: false, message: 'Admin permission required' });
  }
  next();
}

router.post('/products', protect, adminOnly, createProduct);
```

Status	Meaning	When to use it
401 Unauthorized	Who are you?	No token, an invalid token, or an expired token
403 Forbidden	I know who you are, but you may not do this	A valid token, but the user's role does not allow the action

2. In-Class Exercises

Exercise 1: CORS & a Web UI (HTML/JS + Fetch API) Wired to a RESTful CRUD API

Objective: Build an Express backend on **port 5001** that serves a Product Management API with CORS enabled, then connect the provided single-page Web UI (HTML/JS, opened from a different origin, e.g. Live Server on port 5500) that uses the Fetch API to list, create and delete products.

Step 1 Create the project and the backend API (server.js)

Terminal

```
mkdir lab4-ex1 && cd lab4-ex1
npm init -y
npm install express cors
```

server.js

```
const express = require('express');
const cors = require('cors');

const app = express();
const PORT = 5001;

// CORS: allow a Web UI on another origin to call this API
app.use(cors({
  origin: '*', // in production, list the exact origins, e.g. 'http://localhost:3000'
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
app.use(express.json());

// Simulated in-memory database
let products = [
  { id: 1, name: 'Laptop Dell XPS 15', price: 35000000, category: 'Electronics' },
  { id: 2, name: 'Logitech MX Master 3S Mouse', price: 2400000, category: 'Accessories' }
];

// GET - list all products
app.get('/api/v1/products', (req, res) => {
  res.json({ success: true, count: products.length, data: products });
});

// POST - create a product
app.post('/api/v1/products', (req, res) => {
  const { name, price, category } = req.body;
  if (!name || !price) {
    return res.status(400).json({ success: false, message: 'Product name and price are required!'
});
  }
  const newProduct = { id: Date.now(), name, price: Number(price), category: category || 'General'
};
  products.push(newProduct);
```

```
  res.status(201).json({ success: true, message: 'Product added successfully', data: newProduct
});
});

// DELETE - remove a product by id
app.delete('/api/v1/products/:id', (req, res) => {
  const productId = Number(req.params.id);
  const exists = products.some(p => p.id === productId);
  if (!exists) {
    return res.status(404).json({ success: false, message: `Product ID ${productId} not found` });
  }
  products = products.filter(p => p.id !== productId);
  res.json({ success: true, message: `Deleted product ID ${productId}` });
});

app.listen(PORT, () => {
  console.log(`Backend API running at: http://localhost:${PORT}`);
});
```

Step 2 Connect the Web client (index.html & app.js)

Download the frontend code here, open it with Live Server and check that it calls http://localhost:5001.

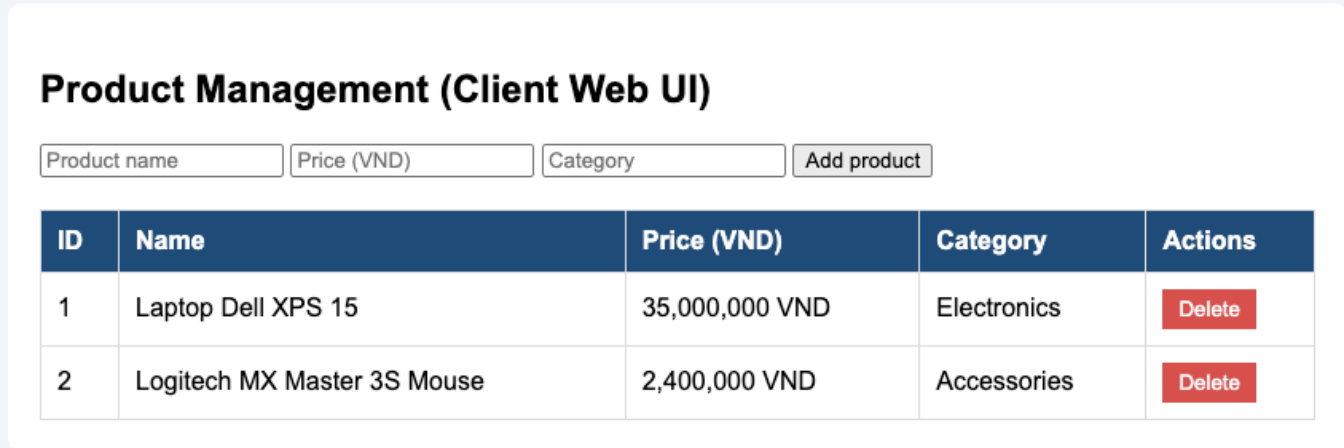


Figure 2.1: The Product Management Web UI

Extended requirements: Update, Filtering & UX Feedback

1. Add an **Update** flow. Backend: implement **PUT /api/v1/products/:id** (400 for invalid data, 404 if the product does not exist). UI: an **Edit** button turns the row into inputs with **Save** / **Cancel**, and Save sends the PUT request.

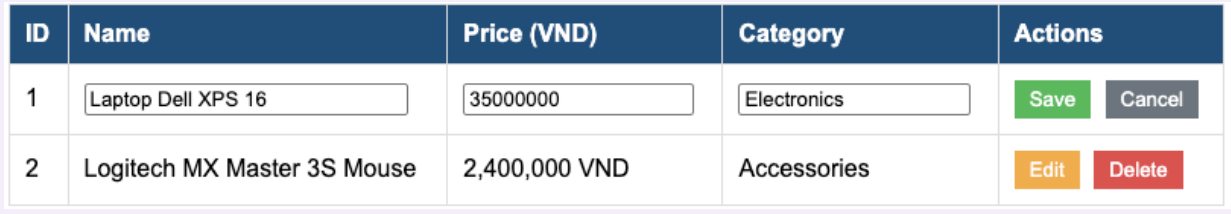


Figure 2.2: Inline editing of a row

2. Add a **category filter** dropdown above the table, filled with the distinct categories in the data. Backend: support **GET /api/v1/products?category=Electronics**. UI: changing the dropdown re-fetches the list with that query.

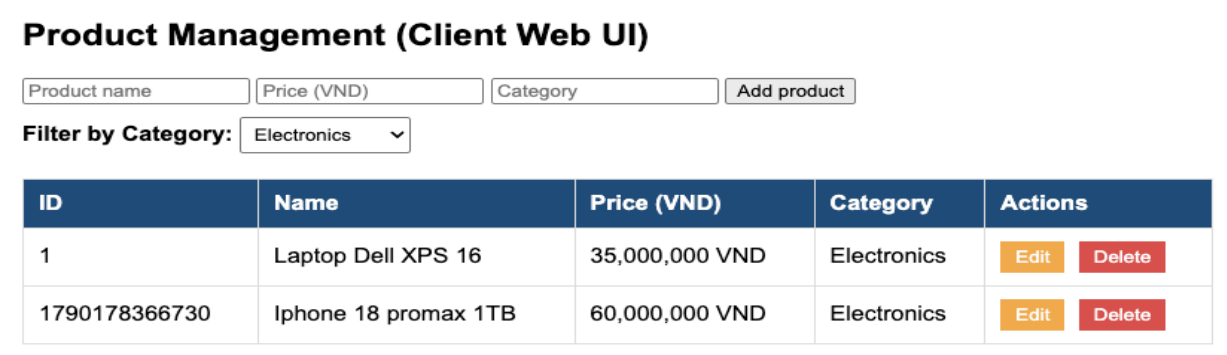


Figure 2.3: Filtering by category

3. Show a visible **error message** in the UI (not only in the Console) whenever a request fails, e.g. when the backend is stopped.

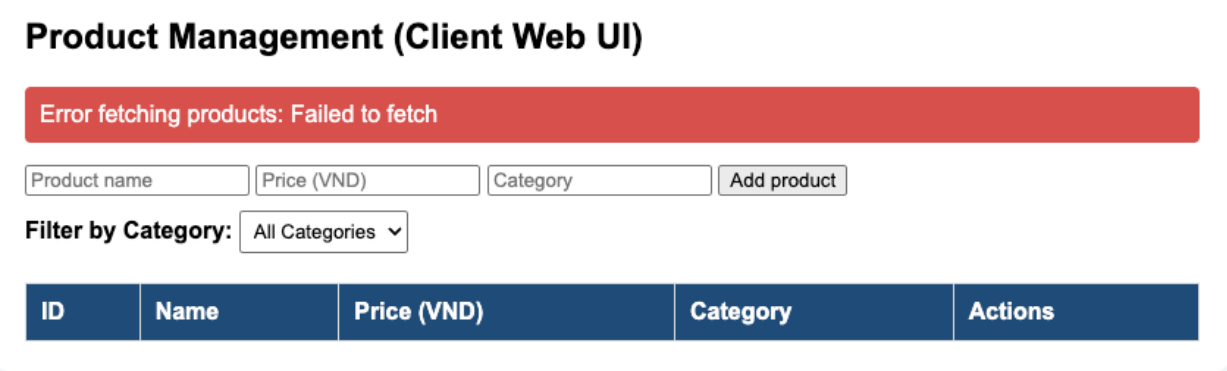


Figure 2.4: Error message shown when the API is unreachable

Exercise 2: Uploading & Downloading Files with Multer

Objective: Configure **multer** to store files in an **uploads/** folder, serve that folder as static files, and connect a Web UI that picks an image and sends it as **FormData** with the Fetch API.

Step 1 Create the project

Terminal

```
mkdir lab4-ex2 && cd lab4-ex2
npm init -y
npm install express cors multer
```

Step 2 Backend upload & static-serving code (upload_server.js)

upload_server.js – setup and static folder

```
const express = require('express');
```

```
const multer = require('multer');
const path = require('path');
const cors = require('cors');
const fs = require('fs');

const app = express();
const PORT = 5001;
app.use(cors());

// Absolute path to the uploads folder; create it if it is missing
const UPLOAD_DIR = path.join(__dirname, 'uploads');
fs.mkdirSync(UPLOAD_DIR, { recursive: true });

// Serve /uploads as static files so the client can view them by URL
app.use('/uploads', express.static(UPLOAD_DIR));
```

upload_server.js – Multer storage, file filter and size limit

```
// Where and under which name Multer saves each file
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, UPLOAD_DIR);
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, file.fieldname + '-' + uniqueSuffix + path.extname(file.originalname));
  }
});

// Only accept JPG / PNG / WEBP images
const fileFilter = (req, file, cb) => {
  const allowedTypes = ['image/jpeg', 'image/png', 'image/webp'];
  if (allowedTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    const error = new Error('Only image files are accepted (JPG, PNG, WEBP)!');
    error.status = 400;
    cb(error, false);
  }
};

const upload = multer({
  storage: storage,
  limits: { fileSize: 2 * 1024 * 1024 }, // 2 MB max
  fileFilter: fileFilter
});
```

upload_server.js – upload route, error handler and start

```
// Accept a single file sent in the form field named 'avatar'
app.post('/api/v1/upload-avatar', upload.single('avatar'), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ success: false, message: 'Please choose an image file!' });
  }
  // Build the public URL from the current host, e.g. http://localhost:5001
  const fileUrl = `${req.protocol}://${req.get('host')}/uploads/${req.file.filename}`;
  res.json({
    success: true,
    message: 'Image uploaded successfully',
    fileInfo: {
```

```
      filename: req.file.filename,
      size: req.file.size,
      mimetype: req.file.mimetype,
      url: fileUrl
    }
  });
});

// Error handler: send Multer errors (file too large, wrong type) as JSON
// instead of Express's default HTML error page
app.use((err, req, res, next) => {
  if (err instanceof multer.MulterError) {
    const message = err.code === 'LIMIT_FILE_SIZE' ? 'The file is too large (max 2 MB)!' :
err.message;
    return res.status(400).json({ success: false, message });
  }
  res.status(err.status || 500).json({ success: false, message: err.message });
});

app.listen(PORT, () => console.log(`Upload server running on port ${PORT}`));
```

Step 3 Connect the client (upload.html) that sends FormData

Download the frontend code here.

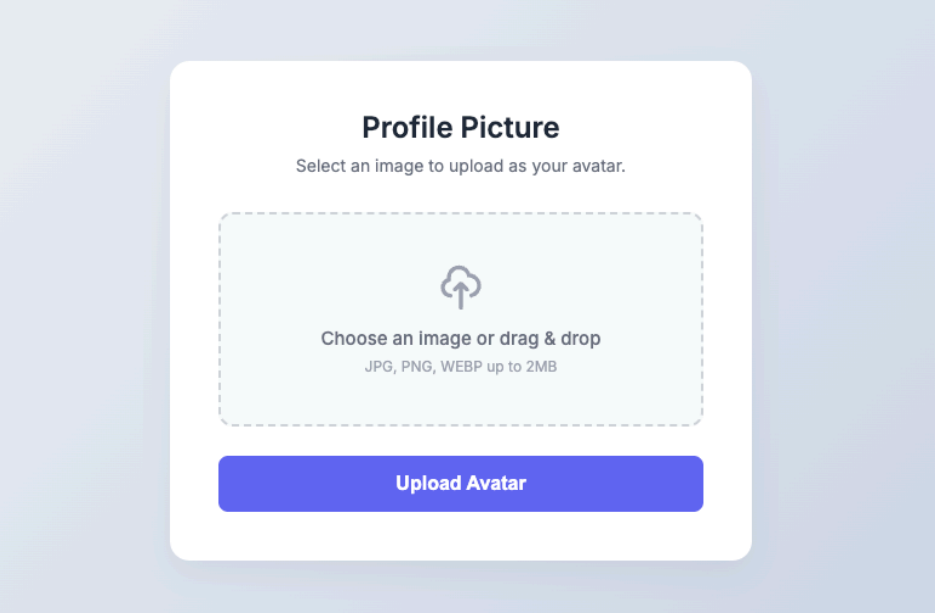


Figure 2.5: The avatar upload page

Extended requirements: Download, Listing & File Management

1. Add **GET /api/v1/download/:filename** that uses **res.download()**, so the browser shows a "Save As" dialog instead of previewing the file (as express.static does).
2. Add **GET /api/v1/files** that lists every file in the **uploads/** folder (use **fs.readdir** and **fs.stat**) and returns its name, size, view URL and download URL.

3. Add **DELETE /api/v1/files/:filename** that removes a file with **fs.unlink** and returns **404** if the file does not exist.
4. Update the Web UI to show a **gallery** of all uploaded files (from the list endpoint), each with a **Download** link and a **Delete** button.

Security tip: use `path.basename(req.params.filename)` before joining it with the uploads folder, so a name like `../server.js` cannot reach files outside **uploads/**.

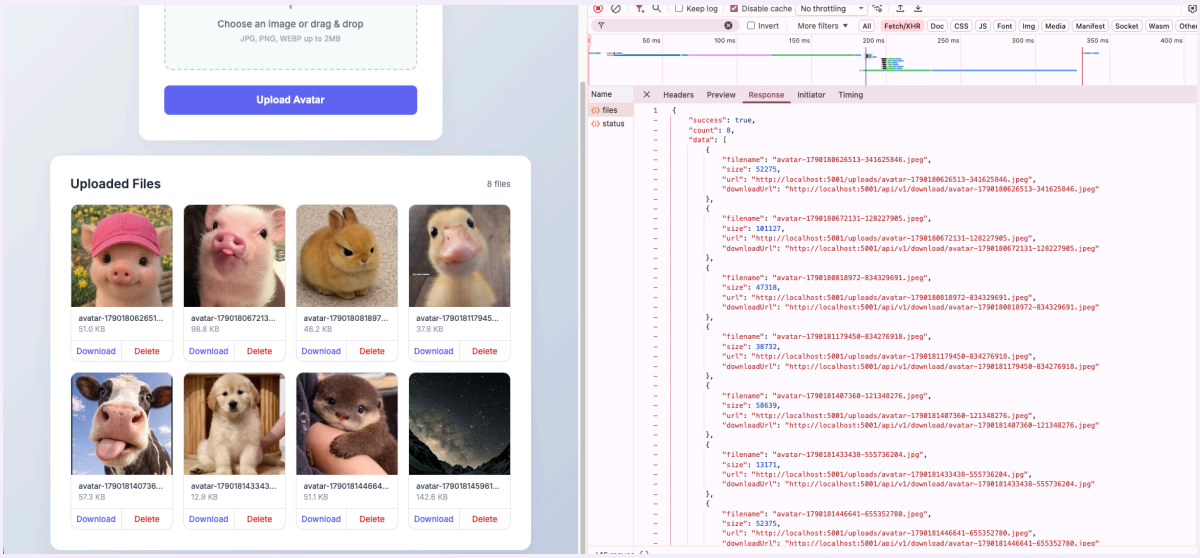


Figure 2.6: The gallery and the GET /api/v1/files response

Exercise 3: Calling an External API from the Backend with Axios

Objective: Build a **GET /api/v1/weather?city=...** endpoint that uses **axios** to call the free **Open-Meteo** API (no key required): first its **geocoding** endpoint to turn a city name into coordinates, then its **forecast** endpoint to get the current weather. Return a simplified JSON response to the client. This is the usual real-world pattern: third-party calls (and any secret keys) live on the server, never in client-side code.

Project structure

```
lab4-ex3/
├── services/weather.service.js
├── controllers/weather.controller.js
├── routes/weather.routes.js
└── server.js
```

Terminal

```
npm init -y
npm install express cors axios
```

Service layer

services/weather.service.js

```
const axios = require('axios');

async function getCoordinatesForCity(city) {
  const response = await axios.get('https://geocoding-api.open-meteo.com/v1/search', {
    params: { name: city, count: 1 },
    timeout: 5000
  });
  const result = response.data.results && response.data.results[0];
  if (!result) {
    const err = new Error(`City "${city}" was not found`);
    err.status = 404;
    throw err;
  }
  return { latitude: result.latitude, longitude: result.longitude, resolvedName: result.name };
}

async function getCurrentWeather(city) {
  const { latitude, longitude, resolvedName } = await getCoordinatesForCity(city);
  const response = await axios.get('https://api.open-meteo.com/v1/forecast', {
    params: { latitude, longitude, current_weather: true },
    timeout: 5000
  });
  const current = response.data.current_weather;
  return {
    city: resolvedName,
    temperatureC: current.temperature,
    windSpeedKmh: current.windspeed,
    observedAt: current.time
  };
}

module.exports = { getCurrentWeather };
```

Controller layer

controllers/weather.controller.js

```
const { getCurrentWeather } = require('../services/weather.service');

exports.getWeather = async (req, res, next) => {
  try {
    const { city } = req.query;
    if (!city) {
      return res.status(400).json({ success: false, message: 'The city query parameter is
required' });
    }
    const weather = await getCurrentWeather(city);
    res.status(200).json({ success: true, data: weather });
  } catch (error) {
    if (error.status === 404) {
      return res.status(404).json({ success: false, message: error.message });
    }
    next(error); // any other error goes to the error handler in server.js
  }
};
```

Routes layer

routes/weather.routes.js

```
const express = require('express');
const router = express.Router();
const { getWeather } = require('../controllers/weather.controller');

router.get('/', getWeather);

module.exports = router;
```

Entry point

server.js

```
const express = require('express');
const cors = require('cors');
const weatherRoutes = require('./routes/weather.routes');

const app = express();
const PORT = 5001;

app.use(cors());
app.use(express.json());
app.use('/api/v1/weather', weatherRoutes);

// Error handler for errors passed with next(error)
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(err.status || 500).json({ success: false, message: 'Could not get the weather right
now' });
});

app.listen(PORT, () => console.log(`Backend API running at: http://localhost:${PORT}`));
```

Test it in Postman or the browser: http://localhost:5001/api/v1/weather?city=Ho Chi Minh

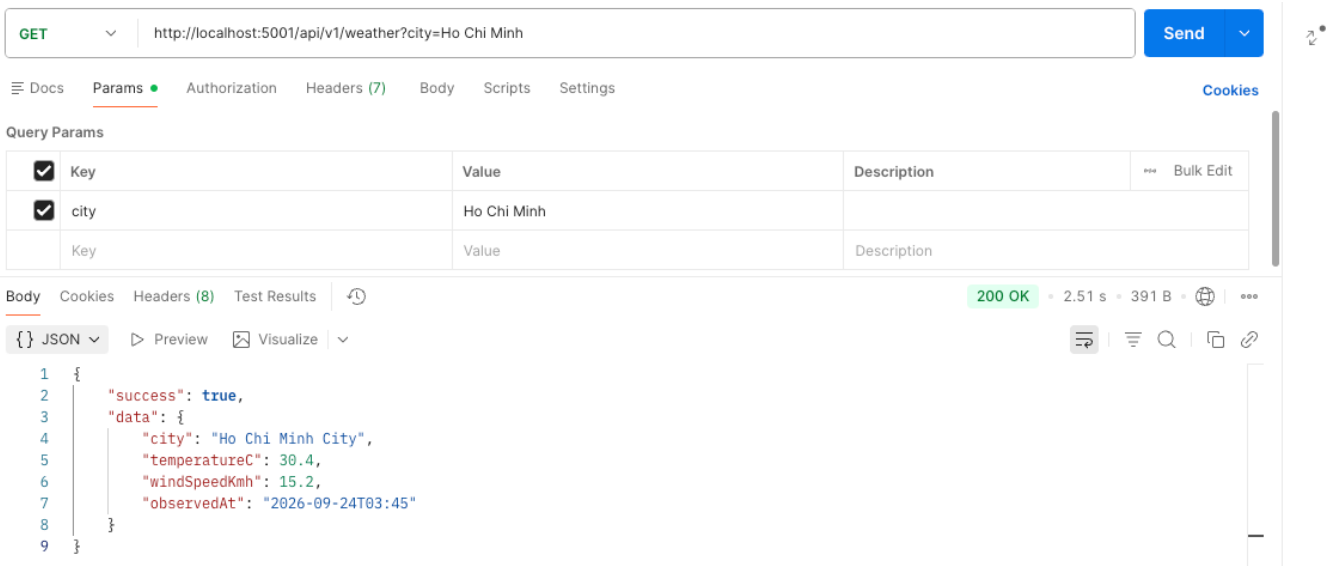


Figure 2.7: Example result in Postman

Extended requirements: Caching, Timeout & Resilience

1. Add a simple in-memory **cache** (a **Map**) keyed by city name with a **5-minute TTL**, so repeated requests for the same city within that time do not call the external API again.

2. Catch a timeout explicitly (**error.code === 'ECONNABORTED'**) and respond with **504 Gateway Timeout** and a friendly message instead of a generic 500. (Tip: set timeout to a very small value such as 1 ms to test it.)

```
{
    "success": false,
    "message": "The weather service is taking too long to respond. Please try again in a moment."
}
```

Figure 2.8: The 504 response on timeout

3. Log how long the external API call took (in milliseconds) for every request, so performance problems are visible in the terminal.

```
Backend API running at: http://localhost:5001
External API call for "Ho Chi Minh" took 3511ms
Cache hit for city: Ho Chi Minh
```

Figure 2.9: Timing and cache logs in the terminal

4. Add a basic **retry**: if the external call fails, retry it **one more time** before returning an error to the client. Do not retry a 404 (city not found): it would fail again.

Exercise 4: Guided Walkthrough – Diagnosing Common API Bugs with DevTools

Objective: Practise recognising the most common real-world API errors with DevTools, so you can spot them instantly later.

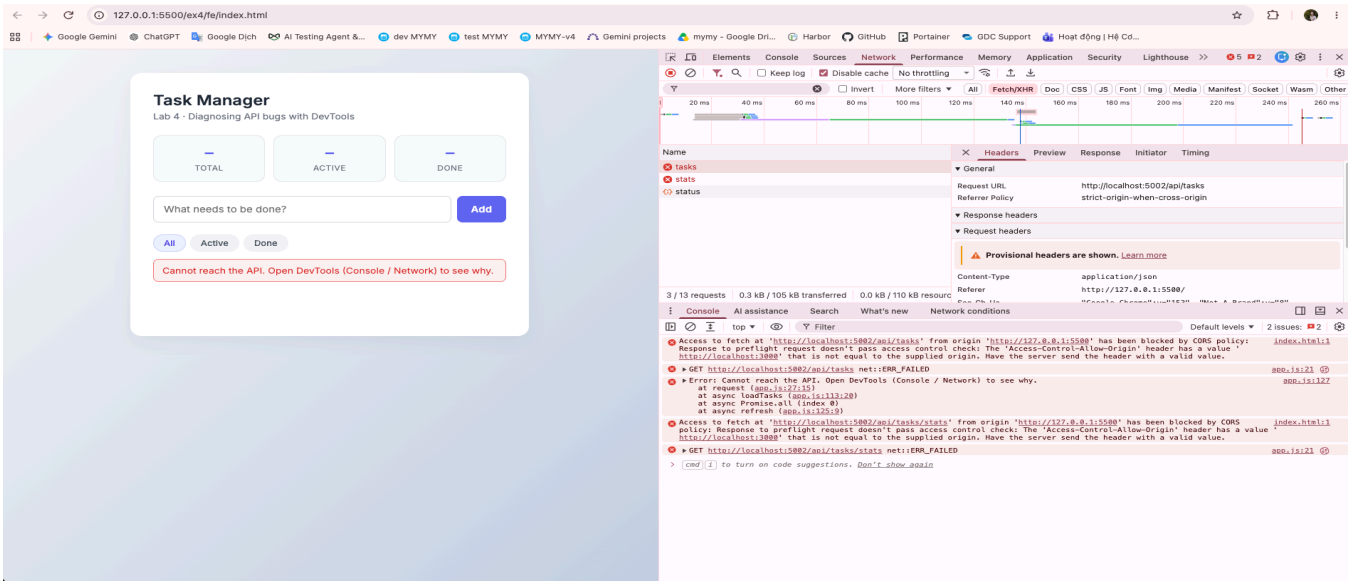


Figure 2.10: The Task Manager app failing with CORS errors in the Console

You receive a project whose **frontend is complete** but whose **backend still has several bugs** (*hint: there are 10*). Use DevTools and what you learned in this lab to find and fix them.

Download the source code here and read the README.md file carefully before you start.

> **What to submit**
>
> A list of the bugs you found. For each bug: **what** you saw (status code, Console message, or wrong data), the **cause**, and your **fix**.
>
> Screenshots showing the app working correctly after all fixes.

3. Homework Exercises

Build the complete **backend** for an e-commerce admin & storefront application. The **frontend is already provided**; your job is the database, the REST API, and the authentication and authorization rules that make every screen in that UI work correctly.

Download the frontend code here.

Requirement 1 – Design the Database (MySQL or MongoDB)

Design a database with 3 entities. You may use MongoDB (Mongoose) or MySQL; the fields are the same either way:

Entity	Fields	Notes
User	id, fullName, age, gender, email, passwordHash, avatarUrl, type	email is unique. type is 'normal' or 'admin'. Never return passwordHash in an API response.
Category	id, name	name is unique.
Product	id, categoryId, name, quantity, price	categoryId references Category.

Requirement 2 – Build the API That Powers the Provided UI

Inspect the provided UI's network calls with DevTools to find the exact endpoints, field names and response shapes it expects, and implement them. Where the UI leaves something open, follow this conventional REST contract:

POST **/api/v1/auth/register** – create an account (fullName, age, gender, email, password, type)

POST **/api/v1/auth/login** – verify credentials and return a JWT

GET **/api/v1/auth/me** – the logged-in user's own profile (protected)

PUT **/api/v1/users/me** – update the logged-in user's own profile (protected)

POST **/api/v1/users/me/avatar** – upload or replace the avatar (protected, multipart/form-data)

GET **/api/v1/categories** – list categories (any logged-in user)

GET **/api/v1/products** – list products (any logged-in user); supports ?categoryId= filtering

POST **PUT** **DELETE** **/api/v1/categories, /api/v1/products** – admin only

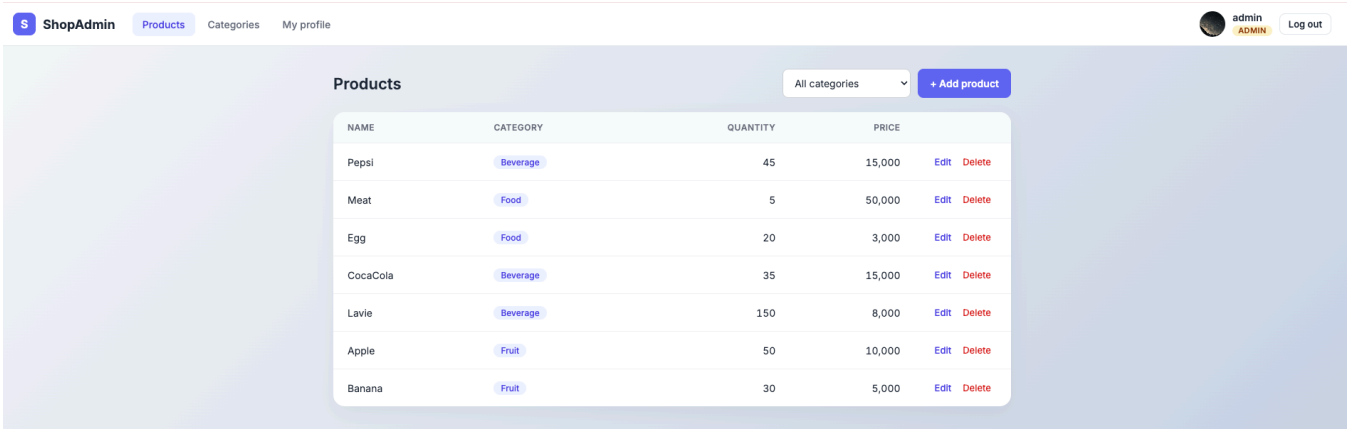


Figure 3.1: The Products screen (admin view)

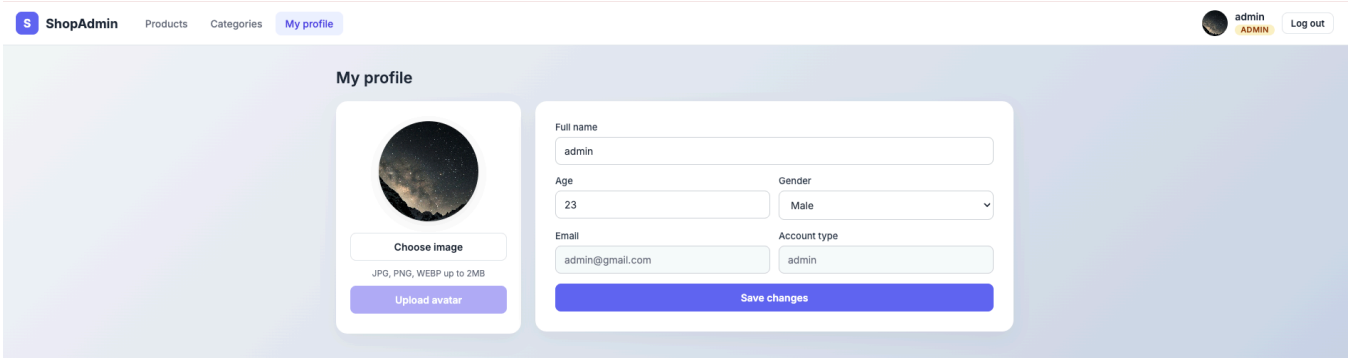


Figure 3.2: The My profile screen

Requirement 3 – Register/Login with a Role, Passwords Hashed with bcrypt

- Registration accepts a **type** of 'normal' or 'admin'. Reject a duplicate email with a clear error (400 or 409).
- Hash the password with **bcryptjs** before saving it, and check it with **bcrypt.compare** at login (section 1.7).

- On success, login issues a **JWT** whose payload contains the user's **id** and **type** (section 1.8). Keep the secret in an environment variable (.env), not in the code.

> **Note**
> Letting users pick 'admin' at sign-up is only to make this lab easy to test. In a real system, admin accounts are created by another admin or directly in the database.

Requirement 4 – Change Avatar (Reusing Multer from Exercise 2)

Reuse the Multer disk-storage pattern from Exercise 2 so a logged-in user can replace their avatar. The route is protected and always updates the user from the token (req.user.id), so users can only change their own avatar. Save the new URL in avatarUrl.

Requirement 5 – Enforce Permissions Server-Side with the JWT

The UI shows the same controls to every user, so the **server** must enforce the rules:

- No token, or an invalid/expired token → **401 Unauthorized**.
- A valid token, but a 'normal' user tries to create, edit or delete a category or product → **403 Forbidden**. The UI then displays that error.

Submission Requirements

- **Source code:** a GitHub repository link (preferred) or a zipped project folder, with a README that explains the setup steps (do not commit node_modules or your .env file).
- **Video demo:** a short screen recording that shows every item of the checklist below, in order.

Video demo checklist

1. Sign up two accounts: one with type 'normal' and one with type 'admin'.
2. Log in as the normal user and show the storefront view.
3. As the normal user, try to create, edit or delete a category or product, and show the resulting **403** error (on screen and/or in DevTools).
4. Change the normal user's avatar and show the change in the UI.
5. Log out, then log in as the admin.
6. As the admin, create, edit and delete a category and a product, showing each change in the UI.
7. Briefly show the JWT itself (e.g. in DevTools > Application > Local Storage, or in a request's Authorization header) to prove the token drives the role check.